

CLAIMS AI COPILOT

RAG-Powered Document Intelligence Platform

Software Design Document
Version 1.0

Status	DRAFT For Review
Version	1.0
Date	February 20, 2025
Author	Vishak Lakshman
Reviewers	Principal Architect · Security Lead · ML Lead · Claims Product Owner

1. Introduction

1.1 Overview

Claims Handlers and Policyholders today must navigate multiple disconnected systems policy management platforms, claims adjudication tools, scanned document repositories, and email archives to answer even routine questions about coverage, exclusions, or claim status. The cognitive overhead is high, error rates are material, and resolution times suffer.

Claims AI Copilot is a production-grade, plug-and-play AI assistant purpose-built to serve the insurance claims and policy domain. Grounded in Retrieval-Augmented Generation (RAG), hosted entirely on AWS, and orchestrated with LangGraph, the system ingests heterogeneous document formats PDFs, EPUBs, scanned images, email threads, structured data exports and makes the knowledge they contain instantly accessible through a natural-language interface.

A reinforcement learning (RL) feedback loop continuously aligns model outputs with user expectations, reducing hallucinations and improving answer precision over time without requiring manual re-training cycles.

1.2 Problem Statement

The following pain points motivate this design:

- Claim Handlers spend an estimated 35-45% of each shift searching across systems for policy terms, coverage limits, and exclusion clauses.
- Policyholders calling support centres wait on hold while agents cross-reference printed or PDF policy booklets.
- Critical information is buried in multi-hundred-page PDFs, scanned legacy documents, and email correspondence that standard keyword search cannot surface accurately.
- Hallucinated or incorrect AI answers in high-stakes insurance contexts create liability and regulatory risk.
- Current tools offer no feedback mechanism to correct or improve answers over time.

1.3 Goals

- Reduce average information retrieval time for Claim Handlers from ~12 minutes to under 60 seconds.
- Provide cited, source-grounded answers that minimise hallucination risk.
- Support all common document input types: PDF, EPUB, scanned image (OCR), plain text, HTML, DOCX, and email (EML/MSG).

- Achieve answer relevance (RAGAS faithfulness) ≥ 0.90 within 90 days of production launch.
- Scale to 10,000+ concurrent sessions without architecture changes.
- Operate within a cost envelope competitive with existing SaaS knowledge-base tools.

1.4 Non-Goals

- This system does not make automated claims decisions or approve/deny claims.
- It does not replace the authoritative policy management system of record.
- Real-time data feeds (live market pricing, live fraud scoring) are out of scope for v1.
- Multi-language support beyond English is deferred to v2.

1.5 Stakeholders

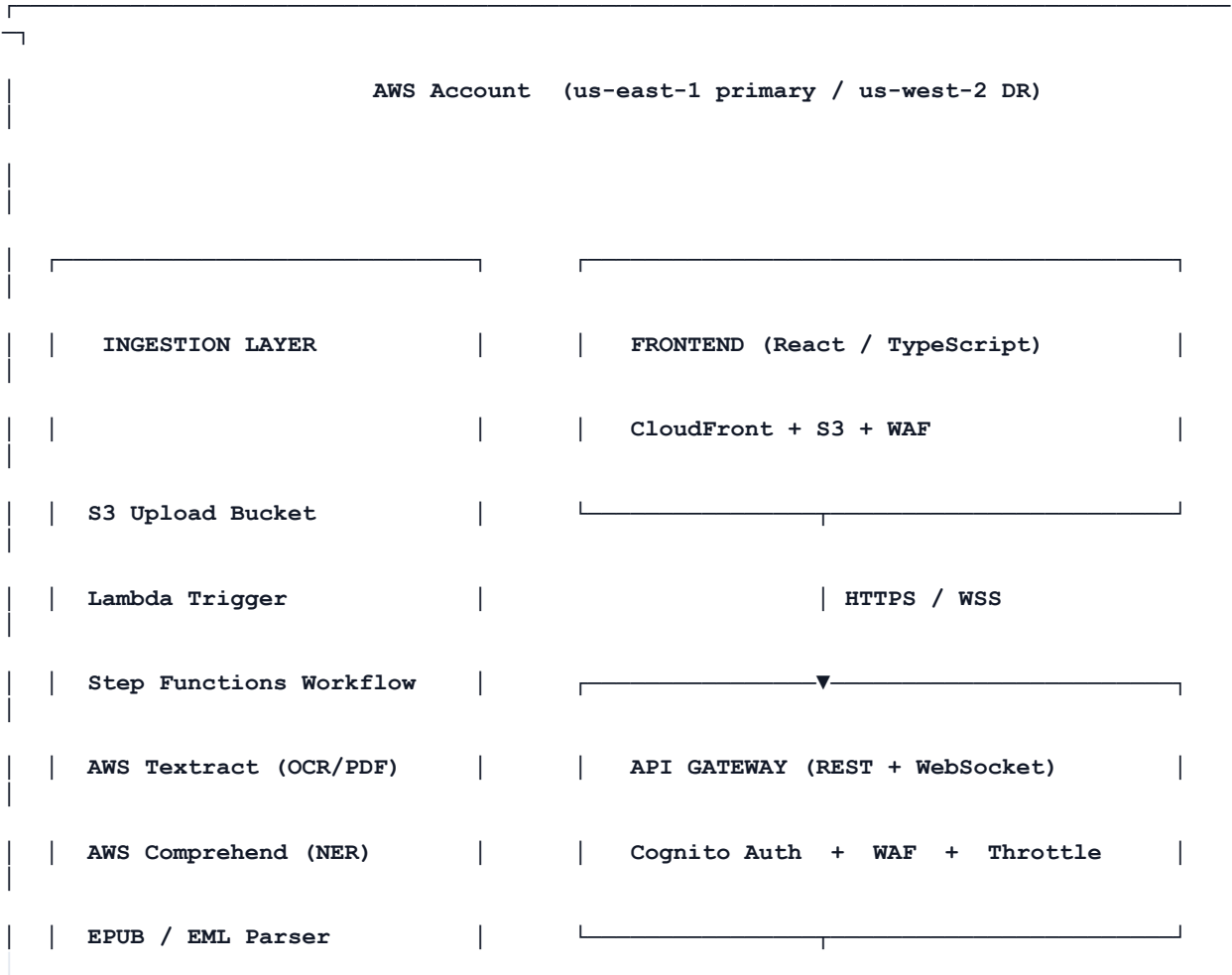
Stakeholder	Role	Interest
Claims Handlers	Primary end-user	Fast, accurate policy/claims answers
Policyholders	Secondary end-user (self-service portal)	Plain-language policy explanations
Claims Operations Lead	Product Owner	Efficiency metrics, SLA compliance
Legal & Compliance	Reviewer	Accuracy, auditability, data privacy
IT Security	Reviewer	Data residency, access control, encryption
ML Engineering	Builder	Model quality, feedback loop, evals
AWS Cloud Ops	Operator	Cost, reliability, scaling

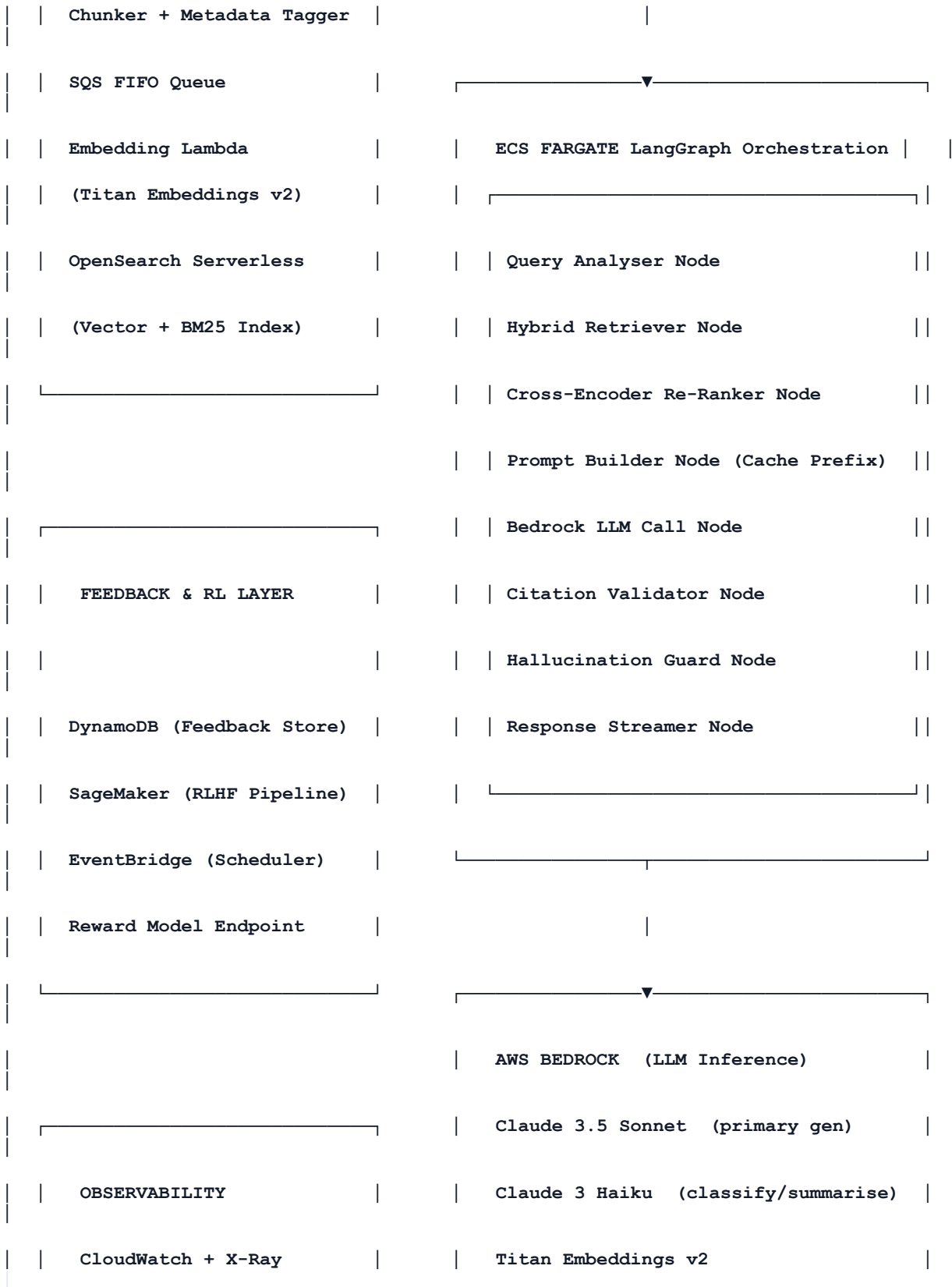
2. System Architecture

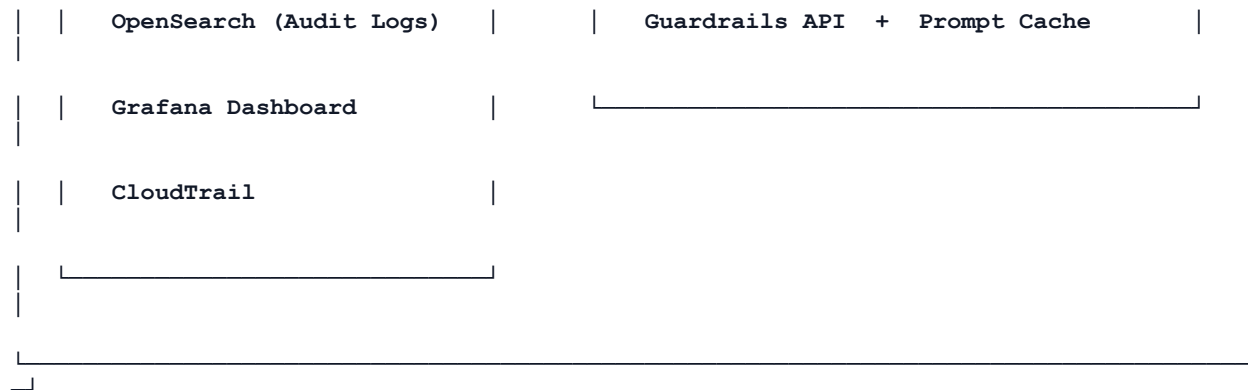
2.1 Architecture Overview

The platform is composed of five logical layers: (1) Ingestion & Document Processing, (2) Embedding & Vector Store, (3) Retrieval & Orchestration, (4) LLM Inference via AWS Bedrock, and (5) Feedback & Continuous Improvement. All layers are deployed within a single AWS organization across two availability zones for resilience.

2.1.1 High-Level Architecture Diagram







2.2 Component Descriptions

2.2.1 Frontend React / TypeScript

A single-page application (SPA) built with React 18 and TypeScript. Key UI components include a conversational chat interface with streaming token display, a document upload panel, source-citation cards linking answers to specific document passages, a thumbs-up / thumbs-down feedback widget, and an admin dashboard for metrics. The SPA is served from Amazon S3 fronted by CloudFront with WAF rules for OWASP Top 10 protection. Authentication is handled by Amazon Cognito with SAML federation to the corporate IdP.

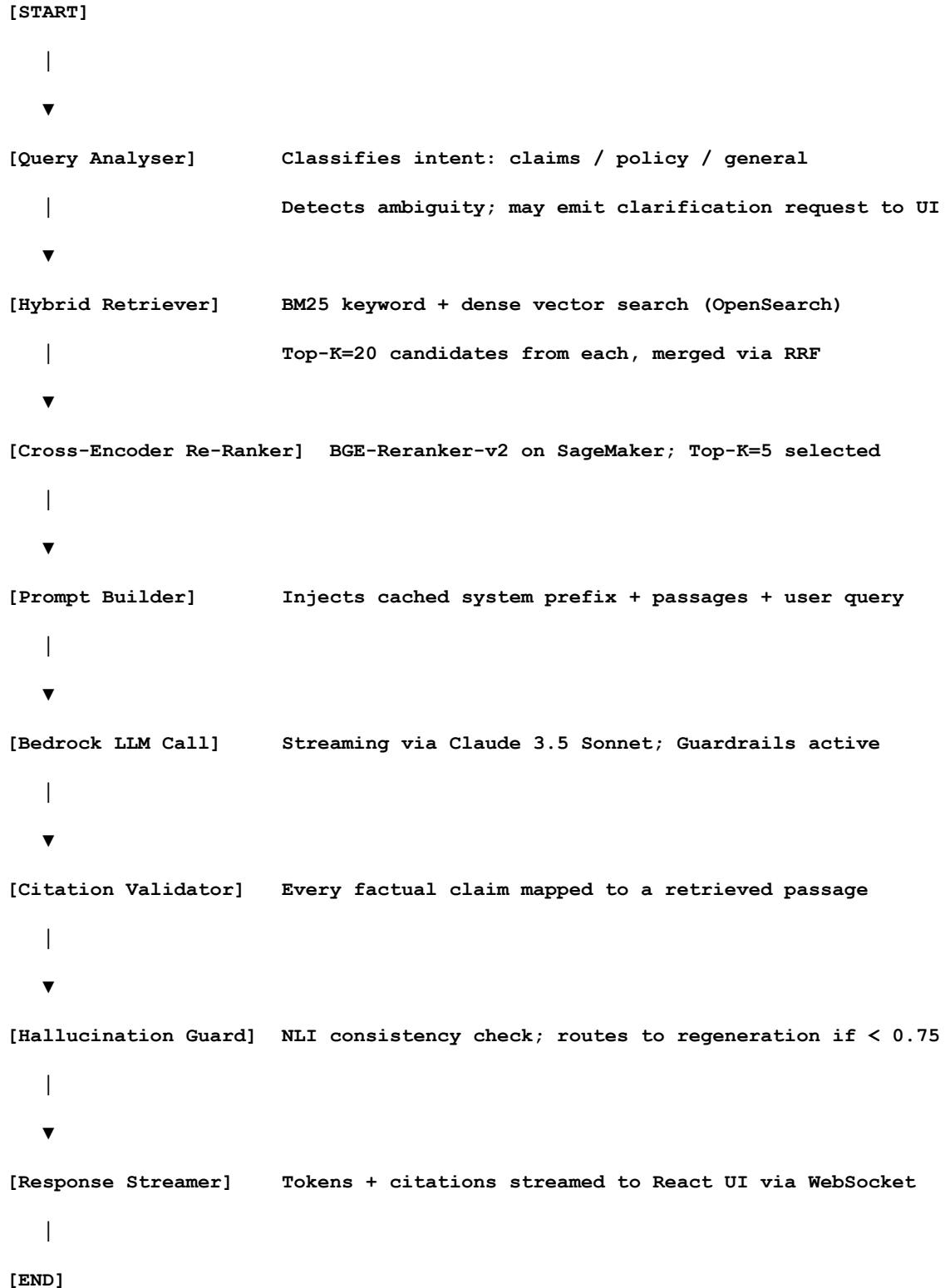
2.2.2 API Gateway & WebSocket

Amazon API Gateway exposes both REST (document upload, user management, feedback submission) and WebSocket endpoints (streaming LLM responses). API Gateway enforces rate limiting (100 req/s per tenant), JWT authorisation via Cognito, and logs all invocations to CloudWatch.

2.2.3 LangGraph Orchestration Service (ECS Fargate)

The core reasoning engine is a Python service on Amazon ECS Fargate. LangGraph models the RAG pipeline as a directed acyclic graph (DAG) of nodes, enabling conditional branching, retry logic, and streaming intermediate state.

LangGraph Node Flow:



2.2.4 Document Ingestion Pipeline

Triggered by S3 object creation events. A Lambda enqueues a processing job to SQS. An ECS worker pool processes jobs through: format detection, text extraction (Textract / custom parsers), semantic chunking, metadata tagging (Comprehend NER), embedding (Titan v2), and indexing (OpenSearch Serverless).

2.2.5 Amazon OpenSearch Serverless Vector + Lexical Index

OpenSearch Serverless provides a fully managed, auto-scaling vector database. The index schema combines k-NN (HNSW) vector fields with standard BM25 keyword fields, enabling hybrid search the most accurate retrieval pattern for domain-specific corpora where exact terminology matters (policy clause numbers, legal definitions).

2.2.6 AWS Bedrock LLM Inference

Amazon Bedrock provides Claude 3.5 Sonnet as the primary generation model and Claude 3 Haiku for lightweight tasks. Bedrock Guardrails blocks PII leakage, profanity, and off-topic responses. Prompt Caching caches static system-prompt prefixes, reducing latency and token cost by up to 90% on repeated invocations.

2.2.7 Feedback & RL Layer

User feedback signals (thumbs up/down, free-text corrections, implicit dwell-time) are stored in Amazon DynamoDB. An EventBridge scheduled rule triggers a nightly SageMaker Pipeline for reward-model scoring, DPO preference-pair construction, and fine-tuning. Model checkpoints are stored in S3 and promoted via a blue/green workflow.

3. Data Flow & Request Lifecycle

3.1 Document Ingestion Flow

User / Admin

| HTTP PUT (pre-signed S3 URL)

▼

S3 Bucket (raw-docs / {tenant-id}/{doc-id})

| S3 Event Notification

▼

Lambda Dispatcher → SQS FIFO Queue (per-tenant) → DLQ (after 3 retries)

|

▼

ECS Ingestion Worker

└ [PDF / Scanned] → Textract → Text + layout JSON

└ [EPUB] → ebooklib → Chapter-level text blocks

└ [Email EML/MSG] → email lib + extract-msg → Body + attachments

└ [DOCX/TXT/HTML] → python-docx / BeautifulSoup → Clean text

|

└ Semantic Chunking (512 tok, 64 overlap)

└ Metadata Extraction (Comprehend NER)

└ Titan Embeddings v2 → 1024-dim vectors

▼

OpenSearch Serverless (k-NN + BM25 Index)

|



DynamoDB (Document Registry: status, metadata, chunk map)

3.2 Query / Answer Flow

Claim Handler / Policyholder (Browser)

| WebSocket connect + JWT



CloudFront → API Gateway (WebSocket) → Cognito Auth

|



LangGraph Service (ECS Fargate)

|

└─ [Node 1: Query Analyser]

| Classify intent, extract entities, detect ambiguity

| If ambiguous: emit clarification to UI, await response

|

└─ [Node 2: Hybrid Retriever]

| Dense search: Titan Embeddings -> AOSS k-NN (top-20)

| Sparse search: BM25 keyword (top-20)

| Reciprocal Rank Fusion -> merged top-30

|

└─ [Node 3: Cross-Encoder Re-Ranker]

| BGE-Reranker-v2 on SageMaker endpoint

```
|      Top-5 passages selected + relevance scores
|
|
├─ [Node 4: Prompt Builder]
|
|      Cached system prefix + passages + query
|
|      Prompt length budget check; truncate if needed
|
|
├─ [Node 5: Bedrock Claude 3.5 Sonnet]
|
|      Streaming generation, Guardrails active
|
|      Temperature=0.1, top-p=0.95
|
|
├─ [Node 6: Citation Validator]
|
|      Every factual sentence -> mapped to source passage
|
|      Unsupported claims flagged or removed
|
|
├─ [Node 7: Hallucination Guard]
|
|      NLI model checks answer entailment vs. retrieved passages
|
|      Score < 0.75  ──>  Regeneration branch (Node 5 retry, max 2x)
|
|      Score < 0.55  ──>  'Insufficient information' response
|
|
└─ [Node 8: Response Streamer]
|
|      Tokens + citations -> WebSocket -> React UI
|
|      Interaction record -> DynamoDB (for RL pipeline)
```

3.3 Feedback Flow

User clicks Thumbs-Up / Thumbs-Down or submits text correction

|

▼

POST /feedback → API Gateway → Lambda → DynamoDB feedback_events

{session_id, query, answer, retrieved_docs, rating, correction, timestamp}

|

▼ (nightly EventBridge cron)

SageMaker Pipeline

└─ Reward Model scores all interactions

└─ Preference pairs (good vs. bad answer) constructed

└─ DPO fine-tune OR Prompt Strategy Update

└─ RAGAS Evaluation Gate (must pass before promotion)

|

▼

Blue/Green Deploy → Production

4. Document Processing & OCR

4.1 Supported Input Types

Format	Extraction Engine	OCR Required	Notes
PDF (text-native)	PyMuPDF / pdfminer	No	Layout preservation
PDF (scanned)	AWS Textract	Yes	Form / table extraction
EPUB	ebooklib + BeautifulSoup	No	Chapter-aware chunking
DOCX / DOC	python-docx	No	Table + heading structure
HTML / HTM	BeautifulSoup4	No	Boilerplate removal
Plain Text / CSV	Native Python	No	
EML / MSG (Email)	email lib / extract-msg	No	Thread-aware parsing
TIFF / PNG / JPEG	AWS Textract	Yes	Scanned policy pages
XLSX / XLS	openpyxl / xlrd	No	Claims data tables
ZIP Archive	zipfile + recursive	Varies	Unpack then process each file

4.2 Chunking Strategy

A fixed-size chunking strategy alone is insufficient for legal/insurance documents because clause boundaries rarely align with token counts. The system uses a two-pass semantic chunking approach:

- Pass 1 Structure-Aware Split: Respect document hierarchy (chapter -> section -> clause -> sub-clause). Textract layout JSON provides heading-level signals; EPUB chapter elements delimit primary splits.
- Pass 2 Token Budget Enforcement: Chunks exceeding 512 tokens are recursively split at sentence boundaries. Chunks below 64 tokens are merged with their neighbour.
- Overlap: 64-token sliding window ensures clause context is not lost at boundaries.
- Parent-Child Indexing: Each chunk stores a parent_chunk_id. Retrieved child chunks can expand to their parent section (Small-to-Big Retrieval).

4.3 OCR Quality & Post-Processing

AWS Textract confidence scores are stored per word. A post-processing Lambda applies:

- Confidence Filtering: Words below 80% confidence are flagged with [UNCERTAIN] markers.
- Dictionary Correction: Insurance domain dictionary used for Levenshtein-distance correction of low-confidence words (e.g. subrogation, deductible, endorsement).
- Table Reconstruction: Textract table JSON is converted to Markdown table syntax before chunking, enabling LLM reasoning over tabular claims data.

5. RAG Design & Retrieval Strategy

5.1 Embedding Model

Amazon Titan Embeddings v2 produces 1024-dimensional vectors with a context window of 8,192 tokens. It is the recommended choice for AWS-native deployments and offers strong performance on legal and financial English text. Embeddings are generated at both ingestion time (document chunks) and query time (user question).

5.2 Hybrid Search

Neither pure dense nor pure sparse retrieval is optimal for insurance text. Dense retrieval excels at semantic similarity but can miss exact clause numbers or defined terms. Sparse (BM25) retrieval excels at exact term matching but misses paraphrased queries. The system implements Reciprocal Rank Fusion (RRF) to merge ranked lists from both modalities. OpenSearch Serverless natively supports both k-NN and BM25 on the same index.

5.3 Re-Ranking

A cross-encoder re-ranker (BGE-Reranker-v2-m3, deployed on a SageMaker real-time endpoint) scores each (query, chunk) pair jointly. The top-5 passages from the re-ranker are passed to the LLM. This two-stage architecture (retrieve many, re-rank few) balances recall and precision cost-effectively.

5.4 Metadata Filtering

Pre-retrieval metadata filters scope searches before the vector search runs, reducing latency and noise:

- `tenant_id`: Strict row-level isolation tenants never see each other's documents.
- `doc_type`: Claim Handler can filter to Policy only or Claims only.
- `effective_date_range`: Filter to policies active on a specific loss date.
- `policy_number`: Targeted lookup for a specific named insured's policy.

5.5 Prompt Caching

AWS Bedrock Prompt Caching allows static portions of the prompt (system instructions, domain persona, output format rules) to be cached on the Bedrock inference fleet. Cache hit rates of 85-95% are expected in production, yielding latency reductions of ~1.5 s per request and cost reductions of ~80% for cached input tokens (billed at ~10% of standard rate).

5.6 Prompt Design

The system prompt follows a structured template separating the cached system prefix from the dynamic context:

SYSTEM (cached prefix):

You are ClaimsIQ, an AI assistant for insurance claims and policy questions.

You ONLY answer based on the provided source passages.

Always cite the source document name and section for every factual claim.

If the answer is not in the passages, say: 'I could not find this information in the available documents. Please contact your Claims Handler.'

Never speculate beyond the provided evidence.

Format: concise paragraphs with inline citations [Doc: {name}, Sec: {x}].

CONTEXT PASSAGES (dynamic):

[1] Source: Policy_ABC123_2024.pdf, Section: 4.2 Fire Coverage

Text: 'Coverage applies to direct physical loss caused by fire...'

[2] ...

USER: {query}

6. Hallucination Reduction Strategy

6.1 Why Hallucinations Are Critical in Insurance

In the claims and policy domain, a hallucinated answer is not merely inconvenient it can constitute misrepresentation of coverage, expose the carrier to errors-and-omissions liability, and cause direct financial harm to policyholders. Hallucination reduction is a first-class engineering requirement, not a post-launch optimisation.

6.2 Multi-Layer Defence

Layer	Mechanism	When Applied
Retrieval Quality	Hybrid search + re-ranking ensures LLM only sees verified document content	Before LLM call
Prompt Engineering	Strict grounding instruction: answer only from passages; cite everything	At prompt build
LLM Guardrails	Bedrock Guardrails: blocks policy-number fabrication, PII generation, off-topic responses	During generation
Citation Validation	Every factual sentence must map to a retrieved chunk (embedding similarity + Jaccard)	After generation
NLI Consistency Check	DeBERTa-v3 NLI checks answer entailment vs. passages. Score < 0.75 triggers regeneration.	After generation
Regeneration Fallback	Up to 2 retries with lower temperature (0.05) and explicit 'do not infer' instruction	On guard failure
Abstention	If both regeneration attempts fail, a canned 'insufficient information' response is returned	On repeated failure
RL Feedback Loop	User corrections feed reward model; poor answer patterns penalised in future fine-tunes	Continuous

6.3 Citation Validation Detail

1. Sentence-tokenise the generated answer (spaCy sentencizer).
2. For each sentence, compute embedding similarity to all 5 retrieved passages.

3. If max similarity < 0.70 AND string overlap (Jaccard) < 0.15, flag sentence as unsupported.
4. Unsupported sentences are removed with a disclaimer appended, or escalate to regeneration if the unsupported proportion exceeds 30%.

6.4 Prompt Regeneration

When the Hallucination Guard fires, the regeneration branch appends an explicit instruction, lowers temperature to 0.05 for more deterministic output, and includes a negative example of the hallucinated sentence. Up to 2 retries are attempted before triggering abstention.

7. Reinforcement Learning Feedback System

7.1 Why RL Feedback Matters

Pre-training and initial RLHF on Claude equip the model with broad instruction-following capability, but do not make it an expert in a specific carrier's policy wordings, claims handling procedures, or internal taxonomy. The RL feedback loop bridges this gap by continuously learning from the judgements of domain experts Claims Handlers who know whether an answer is correct and complete.

7.2 Feedback Signal Types

Signal Type	Description & Weight
Thumbs Up (explicit)	High-confidence positive signal. Weight: 1.0
Thumbs Down (explicit)	High-confidence negative signal. Weight: 1.0
Free-text Correction	User provides correct answer. Highest quality training signal. Weight: 2.0
Implicit: Copy to Clipboard	User copied the answer likely positive. Weight: 0.3
Implicit: Session Abandonment	User left immediately after answer likely negative. Weight: 0.4
Implicit: Follow-up Clarification	User had to ask again partial negative. Weight: 0.5
Escalation to Human Agent	User escalated call strong negative signal. Weight: 0.8

7.3 Reward Model

A reward model (RM) is trained on the labelled preference dataset. The RM is a DeBERTa-v3-base model fine-tuned with a regression head on the feedback dataset, producing a scalar reward score for any (query, context, answer) triple. The RM is retrained weekly as feedback volume grows.

7.4 RLHF / DPO Training Pipeline

`EventBridge` (nightly trigger)

|



SageMaker Pipeline

|

└─ [Step 1] Load feedback_events from DynamoDB

|

Filter: sessions with explicit thumbs-up or thumbs-down

|

└─ [Step 2] Construct Preference Pairs

|

{ prompt, chosen: good_answer, rejected: bad_answer }

|

└─ [Step 3] Reward Model Scoring

|

Score all interactions; augment with RM-scored pairs

|

└─ [Step 4] DPO Fine-Tune (if ≥ 200 preference pairs)

|

OR Prompt Strategy Update (if < 200 pairs)

|

└─ [Step 5] Evaluation Harness

|

RAGAS Faithfulness ≥ 0.90

|

Answer Relevance ≥ 0.85

|

Hallucination Rate $\leq 2\%$

|

Regression: 150 golden Q&A pairs must all pass

|

└─ [Step 6] Gate: PASS $\rightarrow$ Promote / FAIL $\rightarrow$ Alert ML team

|

└─ [Step 7] Artifact Registry: checkpoint $\rightarrow$ S3, metrics $\rightarrow$ CloudWatch

7.5 Avoiding Feedback Gaming

- Rate limiting: Maximum 10 feedback events per session per hour.
- Outlier detection: Feedback patterns deviating > 3 sigma from per-user baseline are flagged for human review.
- Blind evaluation: A held-out golden test set (hand-curated by claims domain experts) is evaluated independently of the RL loop to catch reward hacking.

8. Model Testing & Accuracy Improvement

8.1 Evaluation Framework RAGAS

The system uses the RAGAS (Retrieval-Augmented Generation Assessment) framework for automated evaluation on a golden dataset of 150+ insurance Q&A pairs curated by Claims Handlers:

Metric	Description	Target	Method
Faithfulness	Fraction of answer claims supported by retrieved context	≥ 0.90	NLI entailment
Answer Relevance	How directly the answer addresses the query	≥ 0.85	LLM-as-judge
Context Precision	Proportion of retrieved chunks that are relevant	≥ 0.80	LLM-as-judge
Context Recall	Fraction of ground-truth evidence found in retrieval	≥ 0.80	String overlap + NLI
Hallucination Rate	% answers containing unsupported factual claims	$\leq 2\%$	Citation validator
Latency P95	Time from query to first token (streaming)	≤ 3 s	CloudWatch
Latency P99	Time from query to complete answer	≤ 8 s	CloudWatch

8.2 Golden Dataset Construction

- Domain experts submit representative queries drawn from actual (anonymised) claim inquiries.
- Reference answers are drafted by senior Claims Handlers and reviewed by Counsel for accuracy.
- Each Q&A pair includes: query, ideal answer, source document references, and difficulty label (simple / moderate / complex).
- Dataset is versioned in S3 and updated quarterly or after major policy form changes.

8.3 A/B Testing

For significant model or retrieval changes, an A/B testing framework routes 10% of production traffic to the candidate system. A/B assignment is deterministic per session

(hash of session_id). Metrics are compared using a Mann-Whitney U test ($\alpha = 0.05$). A candidate is promoted only if it achieves statistically significant improvement on at least two primary metrics and no regression on any metric.

8.4 Accuracy Improvement Flywheel

Production Traffic

|



User Feedback (thumbs-up, thumbs-down, corrections)

|



Reward Model Scoring

|



DPO / Prompt Optimisation

|



RAGAS Evaluation Gate

| PASS



Promoted to Production → (loop)

9. Security Architecture

9.1 Threat Model Summary

The system handles **Personally Identifiable Information (PII)**, **Protected Health Information (PHI in health lines)**, and **proprietary policy wordings**. **Key threats: data exfiltration, prompt injection via malicious document content, cross-tenant data leakage, and model output misuse.**

9.2 Identity & Access Management

- Amazon Cognito User Pools authenticate end-users; SAML 2.0 / OIDC federation with corporate Active Directory.
- RBAC roles ClaimsHandler, Policyholder, Supervisor, Admin, MLEngineer each with minimum-privilege IAM policies.
- Service-to-service authentication uses IAM roles (no long-lived credentials). AWS Secrets Manager holds third-party API keys.
- MFA enforced for Admin and MLEngineer roles.

9.3 Data Encryption

Layer	Encryption Approach
Data at rest (S3)	SSE-KMS with customer-managed keys (CMK) per tenant. Annual key rotation.
Data at rest (DynamoDB)	AWS-managed CMK. Field-level encryption for PII columns.
Data at rest (OpenSearch Serverless)	Encryption at rest with CMK. Index-level encryption.
Data in transit	TLS 1.3 enforced everywhere. HSTS preloading on CloudFront.
Bedrock API calls	All calls over TLS within AWS VPC. No data retained by Bedrock after inference.

9.4 Network Security

- All backend services run within a VPC. No public IPs on ECS tasks or OpenSearch clusters.

- AWS WAF rules: OWASP Top 10, rate limiting (100 req/s per IP), SQL injection / XSS / path traversal patterns.
- VPC Endpoints for S3, DynamoDB, Bedrock, SageMaker traffic never traverses the public internet.
- AWS Shield Standard on CloudFront for DDoS mitigation.

9.5 Prompt Injection Defence

Because the system ingests arbitrary document content included in prompts, prompt injection is a primary attack surface.

- Bedrock Guardrails: Configured to detect and block instruction-override patterns.
- Input sanitisation: Document content is HTML-escaped and bracketed in explicit delimiters. The LLM is instructed that delimiter-bounded content is untrusted data.
- Privilege separation: The LLM cannot trigger any tool or API call directly. All tool calls are mediated by the LangGraph layer, which validates parameters.
- Output monitoring: Generated responses scanned for anomalous patterns (system prompt leakage, instruction echoing) before being returned to the user.

9.6 Tenant Isolation

- Every document, chunk, and feedback record carries a tenant_id partition key.
- OpenSearch queries include a mandatory filter on tenant_id enforced at the service layer cannot be overridden by user input.
- IAM resource-based policies on S3 buckets enforce tenant prefix isolation.

9.7 Audit & Compliance

- AWS CloudTrail: All API calls logged to an immutable S3 bucket in a separate audit account.
- Application audit log: Every query, retrieved documents, generated answer, and user feedback stored in OpenSearch. Retention: 7 years for insurance regulatory compliance.
- PII Detection: Amazon Comprehend PII detector runs on all ingested documents and generated responses. Detected PII is masked in logs.
- SOC 2 Type II / HIPAA: Application-layer controls mapped to SOC 2 CC controls; AWS infrastructure inherits Bedrock and underlying service certifications.

10. Scalability & Cost Optimisation

10.1 Scalability Design

Component	Scaling Strategy
CloudFront + WAF	Global CDN scales to millions of requests automatically
API Gateway	Serverless scales to 10,000 concurrent WebSocket connections per region
ECS Fargate (LangGraph)	Target-tracking auto-scaling on CPU/memory. Min: 2 tasks, Max: 200. Scale-out in <90 s.
SQS + ECS Ingestion Workers	SQS queue depth drives ECS worker scale. Max: 50 tasks; ~500 docs/min throughput.
OpenSearch Serverless	Auto-scales OCU capacity units based on indexing and search load. No ops required.
DynamoDB	On-demand capacity mode scales to any throughput without pre-provisioning.
AWS Bedrock LLM	Serverless inference. Provisioned Throughput available for predictable high-volume workloads.
SageMaker Re-Ranker	Auto-scaling endpoint. Min: 1 instance, Max: 10. Cooldown: 60 s.

10.2 Cost Optimisation

- Prompt Caching: 85-95% cache hit rate reduces Claude input token costs by ~80% for cached tokens.
- Tiered Models: Claude 3 Haiku used for classification and summarisation; Claude 3.5 Sonnet for final answer generation only.
- OpenSearch Serverless: Pay per OCU-hour. Off-peak automatically scales to minimum OCUs.
- S3 Lifecycle Policies: Raw documents transition to S3 Glacier after 90 days.
- Spot Instances for SageMaker Training: DPO fine-tuning uses Managed Spot Training (up to 70% discount).
- Reserved Capacity: Fargate Compute Savings Plans for baseline tasks (20-30% saving).

10.3 Estimated Monthly Cost (1,000 Active Users)

Service	Usage Estimate	Unit Cost	Est. Monthly
---------	----------------	-----------	--------------

Bedrock Claude 3.5 Sonnet	5M output tokens	\$15 / 1M output	~\$75
Bedrock Titan Embeddings v2	50M tokens	\$0.02 / 1K tokens	~\$1,000
Bedrock (cached input tokens)	200M cached tokens	\$0.15 / 1M	~\$30
OpenSearch Serverless	4 OCU average	\$0.24 / OCU-hr	~\$700
ECS Fargate (LangGraph)	4 vCPU average	\$0.04 / vCPU-hr	~\$120
AWS Textract (OCR)	10K pages / month	\$0.0015 / page	~\$15
DynamoDB (on-demand)	10M r/w units	\$1.25 / M units	~\$25
S3 + CloudFront	500 GB transfer	~\$0.09 / GB	~\$45
SageMaker Re-Ranker Endpoint	1x ml.g4dn.xlarge	\$0.736 / hr	~\$530
Total Estimate			~\$2,540 / month

11. Observability & Monitoring

11.1 Key Metrics

- Answer Quality: RAGAS faithfulness (7-day rolling), hallucination rate, citation coverage
- Retrieval: P50/P95/P99 retrieval latency, re-ranker latency, k-NN recall@5
- LLM Inference: Bedrock token usage, prompt cache hit rate, generation latency, Guardrail block rate
- User Signals: Daily active users, thumbs-up rate, thumbs-down rate, escalation rate, session length
- Infrastructure: ECS task count, CPU/memory utilisation, SQS queue depth, error rates

11.2 Alerting

Alert	Threshold & Action
Hallucination Rate spike	> 5% in 1-hour window -> PagerDuty P1, auto-rollback triggered
Thumbs-down rate spike	> 30% in 30-min window -> PagerDuty P2, ML team notified
LLM latency P95 breach	> 5 s for 5 min -> Auto-scale ECS, notify ops
SQS queue depth	> 1,000 messages -> Scale ingestion workers
Guardrail block rate	> 10% -> Security team alert, review WAF logs
RAGAS faithfulness drop	< 0.85 on daily eval -> ML team alert, freeze RL deployment

11.3 Distributed Tracing

AWS X-Ray traces every request end-to-end with custom segments for each LangGraph node. Trace IDs are propagated through the WebSocket connection so individual user sessions can be debugged in the X-Ray Service Map.

12. Deployment & Infrastructure as Code

12.1 IaC AWS CDK (TypeScript)

All infrastructure is defined as code using AWS CDK v2 in TypeScript. Stacks are organised as: NetworkStack, StorageStack, IngestionStack, OrchestrationStack, FrontendStack, ObservabilityStack, and MLStack. Each stack is independently deployable and composable.

12.2 CI/CD Pipeline

Developer pushes -> GitHub Pull Request

|



GitHub Actions: lint + unit tests + RAGAS regression (150 golden pairs)

| PASS



CDK Diff -> Code Review

| Approved



CDK Deploy -> Staging (us-east-1)

|



Integration Tests + Load Test (k6, 500 virtual users, 10 min)

| PASS



Manual Approval Gate (Claims Operations Lead)

|



CDK Deploy -> Production (us-east-1 primary + us-west-2 DR)

ECS Rolling Update: 10% canary -> 5-min bake -> full cutover

|



CloudWatch canary metrics check -> confirmed healthy

13. Future Improvements

13.1 Roadmap

Feature	Value	Effort	Target
Multi-language support (ES, FR, DE)	Serve multilingual policyholders	Medium	Q3 2025
Voice interface (Transcribe + Polly)	Phone channel support for call centres	Medium	Q3 2025
Guidewire / Duck Creek live data integration	Answer questions requiring live claim status	High	Q4 2025
Agentic Workflows (LangGraph multi-agent)	Auto-draft coverage denial letters, summarise claim files	High	Q4 2025
Knowledge Graph (Amazon Neptune)	Model relationships: policy -> endorsement -> exclusion -> clause	High	Q1 2026
Hypothetical Document Embeddings (HyDE)	Improve recall for abstract queries	Low	Q2 2026
Domain-specific Embedding Fine-tune	Replace Titan with fine-tuned model for better insurance recall	High	Q2 2026
Personalised Answers (per-claim context)	Answers tailored to the specific insured's open claim file	Medium	Q3 2026
EU AI Act / NAIC Compliance Module	Explainability reports, bias audits, regulatory attestations	Medium	Q3 2026
Real-time Streaming Ingestion (Kinesis)	Ingest email and system events in < 30 s	Medium	Q4 2026

13.2 Known Limitations (v1)

- Re-ranker latency: BGE-Reranker adds ~400 ms. Future: distilled model or Bedrock-native re-ranking.
- English-only: Titan Embeddings v2 performs best on English. Multilingual model required for expansion.
- Table reasoning: Complex multi-column scanned tables remain challenging. Future: table-to-text structured reasoning.
- Cold-start: First query after ECS scale-out incurs ~5 s latency. Mitigated by warm minimum task count.

14. Appendix

A. Technology Stack Summary

Category	Technology	Justification
Frontend	React 18 + TypeScript + Tailwind CSS	Type safety, fast iteration, strong ecosystem
API	Amazon API Gateway (REST + WebSocket)	Managed, serverless, native Cognito integration
Orchestration	LangGraph (Python) on ECS Fargate	Stateful DAG-based RAG, native streaming, retry
LLM	AWS Bedrock Claude 3.5 Sonnet / Haiku	Managed, prompt caching, Guardrails, no GPU ops
Embeddings	Amazon Titan Embeddings v2	AWS-native, no data egress, strong performance
Vector DB	Amazon OpenSearch Serverless	Hybrid search, managed, auto-scaling, no ops
OCR	AWS Textract + custom parsers	Best-in-class OCR, forms, tables
NER / PII	AWS Comprehend	Managed NLP, PII detection, entity extraction
Auth	Amazon Cognito + SAML federation	Managed, corporate IdP integration, MFA
IaC	AWS CDK v2 (TypeScript)	Type-safe, Git-tracked, multi-stack
CI/CD	GitHub Actions + CDK Pipelines	Native PR integration, parallel stages
Monitoring	CloudWatch + X-Ray + Grafana	End-to-end trace, business metrics
ML Training	Amazon SageMaker Pipelines	Managed ML workflow, Spot training
Secrets	AWS Secrets Manager	Automatic rotation, IAM-gated access

B. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation LLM answers grounded in retrieved documents

LangGraph	Graph-based LLM orchestration library built on LangChain; models pipelines as DAGs
RLHF	Reinforcement Learning from Human Feedback aligning model outputs to human preferences
DPO	Direct Preference Optimisation efficient alternative to PPO-based RLHF fine-tuning
RAGAS	Retrieval Augmented Generation Assessment automated RAG evaluation framework
NLI	Natural Language Inference determining if one text entails or contradicts another
BM25	Best Match 25 probabilistic sparse text retrieval algorithm
HNSW	Hierarchical Navigable Small World efficient approximate nearest-neighbour graph algorithm
RRF	Reciprocal Rank Fusion method for combining ranked retrieval result lists
OCU	OpenSearch Capacity Unit billing unit for OpenSearch Serverless
CMK	Customer Managed Key KMS key owned and rotated by the customer organisation
AOSS	Amazon OpenSearch Serverless
DLQ	Dead Letter Queue SQS queue holding messages that failed all processing retries
HyDE	Hypothetical Document Embeddings generates a hypothetical answer to improve retrieval

C. References

- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Es et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217.
- Rafailov et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS.
- AWS Documentation Bedrock Prompt Caching, Guardrails, Custom Models (2024).
- LangGraph Documentation State Machines, Streaming, Human-in-the-Loop (2024).
- NIST AI RMF Artificial Intelligence Risk Management Framework (2023).

— End of Document —